Preparação do Servidor de IA: do Sistema Operacional à Entrega

Visão geral

Em cerca de oito dias úteis, a equipe de infraestrutura transforma a máquina montada em um servidor Ubuntu seguro, com a GPU utilizável dentro de containers. Nesse ponto, a equipe de LLM assume.

Começa com: máquina montada, ligada ao nobreak e à rede, com GPU de 24 GB e 128 GB de RAM.

Termina com: sistema instalado e protegido, driver NVIDIA fixado, Docker com acesso à GPU, testes de carga aprovados, monitoramento ativo, contas da equipe de LLM criadas e entrega formal assinada.

Fora do escopo: montagem física; modelos, vLLM, gateway e interface (equipe de LLM).

Camadas do servidor e fronteira entre as equipes (de cima para baixo):

Camada	Responsável	Etapa
Modelos, vLLM, gateway e interface	Equipe de LLM	—
Entrega: GPU utilizável em container, contas SSH, /srv/ia, portas liberadas	Ambas	9
Monitoramento, contas da equipe de LLM e backup	Infraestrutura	8
Docker + NVIDIA Container Toolkit	Infraestrutura	6
Driver NVIDIA fixado	Infraestrutura	5
SSH por chave, firewall, atualizações automáticas	Infraestrutura	4
Ubuntu Server LTS, rede, disco de dados em /srv	Infraestrutura	1 a 3
Pré-requisito já pronto: máquina montada, BIOS ajustada, nobreak e rede	—	—

Os testes de carga da Etapa 7 validam todas as camadas antes da entrega.

Equipe:

Papel	Responsabilidade	Etapas
Administrador Linux (executor)	Instala e configura o sistema, driver, Docker, monitoramento e backup	1 a 9
Responsável por rede e segurança	IP, DNS, regras de firewall da empresa, VPN, revisão de acessos	1, 3, 4 e 8
Revisor técnico	Segunda pessoa que confere cada etapa e o runbook	4, 7 e 9
Ponto focal da equipe de LLM	Informa versões e portas necessárias; aceita a entrega	1 e 9
Coordenador	Cronograma e comunicação com a diretoria	Todas

Convenções usadas neste documento: servidor ia-srv01, domínio empresa.local, rede interna 10.0.10.0/24, administrador admin.infra. Substitua pelos valores reais da empresa.

Etapa 1 – Preparação

Antes de iniciar o instalador, a equipe junta todas as informações e confere a BIOS. Isso evita reinstalar por causa de um IP errado ou do Secure Boot bloqueando o driver.

1. Informações a reunir

Informação	Quem fornece	Exemplo
Versão do Ubuntu Server LTS	Ponto focal de LLM	24.04 LTS
Versão mínima de driver NVIDIA/CUDA	Ponto focal de LLM	Conforme as imagens que será utilizado
Portas que os serviços vão expor	Ponto focal de LLM	Ex.: 443 para o gateway
Contas a criar e nível de acesso	Ponto focal de LLM	Nomes, chaves SSH públicas
IP fixo, máscara, gateway, DNS	Rede e segurança	10.0.10.50/24
Nome no DNS interno	Rede e segurança	ia-srv01.empresa.local
Servidor de horário (NTP)	Rede e segurança	Controlador de domínio ou a.ntp.br
Proxy de saída, se houver	Rede e segurança	Necessário para apt e Docker

2. Mídia de instalação

- Baixar a ISO do Ubuntu Server LTS do site oficial e conferir o SHA256:

```
sha256sum ubuntu-24.04.x-live-server-amd64.iso
# comparar com o arquivo SHA256SUMS do site oficial
```

- Gravar em pendrive de 8 GB ou mior com balenaEtcher, Rufus (modo GPT/UEFI) ou dd.

3. Checagem rápida da BIOS

Item	Valor esperado
Modo de boot	UEFI (sem CSM/Legacy)
Above 4G Decoding e Resizable BAR	Ativados
Secure Boot	Desativado, ou ativado com a senha MOK anotada para a Etapa 5
Restore on AC Power Loss	Power On
Vídeo principal	Gráfico integrado, se existir; senão, a própria GPU
Ordem de boot	Pendrive primeiro, só durante a instalação

Entregável: formulário de informações preenchido e pendrive conferido.

Etapa 2 – Instalação do Ubuntu Server

O instalador leva cerca de 30 minutos. As duas escolhas que mais importam: não instalar o Docker pelo snap e não deixar o instalador escolher o driver NVIDIA sozinho.

Tela a tela

1. **Boot:** iniciar pelo pendrive e escolher *Try or Install Ubuntu Server*.
2. **Idioma:** English. Mensagens de erro em inglês são mais fáceis de pesquisar.
3. **Teclado:** Portuguese (Brazil), variante ABNT2.
4. **Tipo de instalação:** *Ubuntu Server* (não a versão *minimized*). Deixar **desmarcado** *Search for third-party drivers*: o driver NVIDIA será instalado de forma controlada na Etapa 5.
5. **Rede:** na placa cabeada, *Edit IPv4* → *Manual*, com os dados da Etapa 1: Subnet 10.0.10.0/24, Address 10.0.10.50, Gateway 10.0.10.1, Name servers com os DNS internos e Search domains empresa.local.
6. **Proxy:** preencher só se a empresa usar proxy de saída.
7. **Mirror:** manter o espelho brasileiro sugerido e aguardar o teste de conexão.
8. **Armazenamento:** *Custom storage layout* no disco do sistema (ver tabela abaixo). **Não tocar no NVMe de dados**, que será configurado na Etapa 3.
9. **Perfil:** nome do administrador, servidor ia-srv01, usuário admin.infra, senha forte guardada no cofre de senhas.
10. **Ubuntu Pro:** pular (pode ser ativado depois, se a empresa desejar).
11. **SSH:** marcar *Install OpenSSH server*. Se o administrador tiver a chave no GitHub, usar *Import SSH identity*; senão, a chave é copiada na Etapa 4.
12. **Featured snaps:** não marcar nada, **principalmente o Docker**. O snap do Docker não funciona bem com o NVIDIA Container Toolkit.
13. Aguardar *Installation complete*, remover o pendrive e escolher *Reboot now*.

Layout do disco do sistema

Partição	Tamanho	Formato	Ponto de montagem
EFI	1 GB	FAT32	/boot/efi
Boot	2 GB	ext4	/boot
LVM (grupo vg-sistema)	Restante do disco	LVM	—
↳ volume lv-root	200 GB	ext4	/
↳ espaço livre no grupo	Restante	—	Reserva para ampliar / ou fazer snapshots LVM

A swap será um arquivo de 16 GB criado na Etapa 3, mais fácil de ajustar que uma partição.

Primeiro login: conferência

```
lsb_release -a        # versão correta do Ubuntu
ip -br addr           # IP fixo aplicado
lsblk                 # disco do sistema e NVMe de dados visíveis
lspci | grep -i nvidia  # GPU reconhecida no barramento
cat /sys/firmware/efi/fw_platform_size   # 64 = boot em UEFI
```

Entregável: sistema instalado, acessível no IP fixo, com a saída da conferência registrada.

Etapa 3 – Configuração inicial

Ao fim desta etapa, o sistema está atualizado, com horário sincronizado, rede definitiva e o NVMe de dados montado em /srv. Duração: meio dia.

1. Rede definitiva

- Impedir que o cloud-init reescreva a configuração de rede depois:

```
sudo touch /etc/cloud/cloud-init.disabled
```

- Conferir /etc/netplan/*.yaml (nome da interface conforme ip -br link):

```
network:
  version: 2
  ethernets:
    enp5s0:
      dhcp4: false
      addresses: [10.0.10.50/24]
      routes:
        - to: default
          via: 10.0.10.1
      nameservers:
        addresses: [10.0.10.2, 10.0.10.3]
        search: [empresa.local]

sudo chmod 600 /etc/netplan/*.yaml
sudo netplan try     # reverte sozinho em 120 s se perder a conexão
```

2. Nome, fuso e horário

```
sudo hostnamectl set-hostname ia-srv01
sudo timedatectl set-timezone America/Sao_Paulo
sudo sed -i 's/^#\?NTP=.*/NTP=10.0.10.2/' /etc/systemd/timesyncd.conf
sudo systemctl restart systemd-timesyncd
timedatectl status   # "System clock synchronized: yes"
```

3. Atualização e pacotes base

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y build-essential linux-headers-$(uname -r) \
  curl wget git vim tmux htop nvtop jq pciutils \
  lm-sensors smartmontools nvme-cli fio stress-ng
sudo reboot
```

Os cabeçalhos do kernel e o build-essential são necessários para o driver NVIDIA compilar seu módulo.

4. Swap de 16 GB e ajustes do kernel

```
sudo fallocate -l 16G /swapfile
sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

cat <<'EOF' | sudo tee /etc/sysctl.d/90-servidor-ia.conf
vm.swappiness = 10
fs.inotify.max_user_watches = 524288
EOF
sudo sysctl --system
```

5. NVMe de dados em /srv

```
lsblk -o NAME,SIZE,MODEL,SERIAL       # confirmar qual é o disco de dados
sudo parted /dev/nvme1n1 --script mklabel gpt mkpart dados xfs 0% 100%
sudo mkfs.xfs -L dados /dev/nvme1n1p1
sudo blkid /dev/nvme1n1p1             # anotar o UUID
echo 'UUID=<uuid> /srv xfs defaults,noatime,nofail 0 2' | sudo tee -a /etc/fstab
sudo mount -a && df -h /srv
sudo mkdir -p /srv/ia /srv/docker /srv/backup
```

O nofail garante que o sistema inicie mesmo se o disco de dados falhar, permitindo o diagnóstico remoto.

Entregável: netplan, timedatectl e df -h conferidos; reinício sem erros.

Etapa 4 – Segurança e acesso remoto

O servidor passa a aceitar apenas login por chave SSH, apenas da rede interna, e recebe correções de segurança sozinho, sem nunca mexer no driver NVIDIA. Duração: meio dia.

1. Chave SSH do administrador

Na estação do administrador:

```
ssh-keygen -t ed25519 -C "admin.infra@empresa"
ssh-copy-id admin.infra@10.0.10.50
ssh admin.infra@10.0.10.50     # testar antes de seguir
```

2. Endurecimento do SSH

```
cat <<'EOF' | sudo tee /etc/ssh/sshd_config.d/90-endurecimento.conf
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
X11Forwarding no
AllowGroups ssh-users
EOF
sudo groupadd -f ssh-users && sudo usermod -aG ssh-users admin.infra
sudo sshd -t && sudo systemctl reload ssh
```

Manter a sessão atual aberta e testar o login em uma segunda janela antes de fechar. O grupo ssh-users define quem pode entrar.

3. Firewall local

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 10.0.10.0/24 to any port 22 proto tcp
sudo ufw enable
sudo ufw status verbose
```

Portas da equipe de LLM são liberadas na Etapa 8, só para a rede interna. Atenção: o Docker publica portas passando por cima do ufw. A regra combinada com a equipe de LLM é publicar serviços apenas em 127.0.0.1 ou atrás de um proxy.

4. Proteção contra força bruta

```
sudo apt install -y fail2ban
cat <<'EOF' | sudo tee /etc/fail2ban/jail.d/sshd.local
[sshd]
enabled = true
maxretry = 5
bantime = 1h
EOF
sudo systemctl enable --now fail2ban
```

5. Atualizações automáticas sem tocar no driver

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
```

Em /etc/apt/apt.conf.d/50unattended-upgrades:

```
Unattended-Upgrade::Package-Blacklist {
  "nvidia-";
```

```
    "libnvidia-";
    "cuda-";
    "docker-ce";
    "containerd.io";
    "nvidia-container-toolkit";
};
Unattended-Upgrade::Automatic-Reboot "false";
```

Reinícios ficam para janelas de manutenção combinadas com a equipe de LLM.

Entregável: login só por chave, ufw ativo, fail2ban ativo, atualizações automáticas com a lista de exclusão conferida pelo revisor.

Etapa 5 – Driver NVIDIA

No host entra apenas o driver NVIDIA. O CUDA Toolkit não é instalado, porque cada container da equipe de LLM traz a própria versão de CUDA. Duração: meio dia.

1. Escolha da versão

```
sudo ubuntu-drivers list --gpgpu
```

- Escolher a linha -server (ramo de produção para uso computacional), na versão que atenda o mínimo informado pela equipe de LLM.
- A versão do driver define a CUDA máxima suportada pelos containers; conferir em nvidia-smi depois da instalação.

2. Instalação

```
sudo ubuntu-drivers install --gpgpu nvidia:<versão>-server
sudo apt install -y nvidia-utils-<versão>-server
sudo reboot
```

Se o Secure Boot estiver ativado: a instalação pede uma senha temporária. No reinício aparece a tela azul *Perform MOK management*: escolher *Enroll MOK* → *Continue* → *Yes* e digitar a mesma senha. Sem isso, o driver não carrega.

3. Verificação

```
nvidia-smi                  # GPU, VRAM, versão do driver e "CUDA Version"
lsmod | grep nvidia         # módulos carregados
lsmod | grep nouveau        # não deve retornar nada
sudo dmesg | grep -iE 'nvrm|xid'  # sem erros
```

4. Modo de persistência

Mantém a GPU inicializada entre usos, evitando atraso na primeira chamada de cada container:

```
sudo systemctl enable --now nvidia-persistenced
nvidia-smi -q | grep -i persistence   # "Enabled"
```

5. Limite de potência (opcional)

Reduzir o limite de potência em 10–20% costuma baixar bastante a temperatura com pouca perda de desempenho. A decisão fica para depois dos testes da Etapa 7.

```
nvidia-smi -q -d POWER          # limites mínimo, padrão e máximo
sudo nvidia-smi -pl <watts>     # vale até o próximo reinício
```

Se adotado, criar um serviço systemd que aplique o limite no boot.

6. Fixação da versão

```
sudo apt-mark hold $(dpkg -l | awk '/^ii/ && /nvidia/ {print $2}')
apt-mark showhold
```

A partir daqui, trocar o driver é uma mudança planejada: janela combinada, fallback avisado e o mesmo roteiro desta etapa.

Entregável: nvidia-smi sem erros, persistência ativa, pacotes fixados e versões registradas na ficha técnica.

Etapa 6 – Docker e NVIDIA Container Toolkit

Esta etapa cria o ponto de contato com a equipe de LLM: um container consegue enxergar e usar a GPU. Duração: meio dia.

1. Docker pelo repositório oficial

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
 https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
 | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
 docker-buildx-plugin docker-compose-plugin
```

2. Configuração do Docker

```
sudo systemctl stop docker docker.socket
cat <<'EOF' | sudo tee /etc/docker/daemon.json
{
 "data-root": "/srv/docker",
 "ip": "127.0.0.1",
 "live-restore": true,
 "log-driver": "json-file",
 "log-opts": { "max-size": "50m", "max-file": "3" }
}
EOF
sudo systemctl start docker
docker info | grep -E 'Docker Root Dir|Logging Driver'
```

Opção	Efeito
data-root	Imagens e volumes no NVMe de dados, não no disco do sistema
ip: 127.0.0.1	Portas publicadas ficam só na máquina, evitando que o Docker fure o ufw
live-restore	Containers continuam rodando se o serviço do Docker reiniciar
log-opts	Logs de containers limitados a 150 MB cada

3. NVIDIA Container Toolkit

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
 | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list \
 | sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' \
 | sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt update
sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

O nvidia-ctk acrescenta o runtime NVIDIA ao daemon.json sem apagar as opções do passo 2; conferir o arquivo depois.

4. Teste da GPU em container

```
sudo docker run --rm --gpus all nvidia/cuda:<tag-compatível>-base-ubuntu24.04 nvidia-smi
```

O mesmo teste em Docker Compose, como a equipe de LLM vai usar:

```
services:
  teste-gpu:
    image: nvidia/cuda:<tag-compatível>-base-ubuntu24.04
    command: nvidia-smi
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]

sudo docker compose up && sudo docker compose down
```

5. Fixação das versões

```
sudo apt-mark hold docker-ce docker-ce-cli containerd.io nvidia-container-toolkit
```

Entregável: nvidia-smi funcionando dentro de container, via docker run e via Compose, com versões registradas.

Etapa 7 – Validação e testes de carga

O servidor só é entregue depois de 24 horas com CPU, memória e GPU em carga máxima, sem erro, travamento ou superaquecimento. Duração: dois dias, quase todo em espera.

1. Registro contínuo da GPU

Deixar rodando durante todos os testes:

```
mkdir -p ~/testes && cd ~/testes
nvidia-smi --query-
gpu=timestamp,temperature.gpu,power.draw,clocks.sm,utilization.gpu,memory.used,clocks_throttle_reasons.a
ctive \
  --format=csv -l 10 > gpu.csv &
```

2. GPU: gpu-burn em container (2 horas)

```
git clone https://github.com/wilicc/gpu-burn.git && cd gpu-burn
sudo docker build -t gpu-burn .
sudo docker run --rm --gpus all gpu-burn ./gpu_burn -tc 7200
```

O resultado precisa terminar com OK. Qualquer FAULTY reprova a GPU.

3. CPU e memória (2 horas)

```
stress-ng --cpu 0 --vm 4 --vm-bytes 80% --timeout 2h --metrics-brief
watch -n 10 sensors          # em outra janela
```

4. Disco de dados

```
sudo smartctl -a /dev/nvme1n1 | grep -iE 'critical|percentage|temperature|error'
sudo fio --name=leitura --filename=/srv/teste.bin --size=20G --rw=read --bs=1M --direct=1
sudo fio --name=escrita --filename=/srv/teste.bin --size=20G --rw=write --bs=1M --direct=1
sudo rm /srv/teste.bin
```

A leitura sequencial importa: é o que define quanto tempo um modelo de dezenas de GB leva para carregar.

5. Estresse combinado (24 horas)

- Em uma sessão tmux, rodar gpu-burn e stress-ng juntos por 24 horas.
- No dia seguinte, verificar gpu.csv, sudo dmesg | grep -iE 'xid|mce|error' e journalctl -p err -b.

6. Testes de reinício

- Reinício normal (sudo reboot): tudo volta sozinho, incluindo montagem de /srv, Docker e persistência da GPU.
- Queda de energia simulada: desligar da tomada com a máquina em uso, religar e confirmar que ela liga sozinha e sobe sem erro.

Critérios de aprovação

Teste	Critério
gpu-burn	Resultado OK, sem erros Xid no dmesg
Temperatura da GPU	Estável e sem redução de clock por temperatura (throttle_reasons) sustentada
CPU e memória	Sem travamento e sem erros MCE

Teste	Critério
Disco	SMART sem alertas; leitura compatível com a especificação do NVMe
24 horas combinadas	Nenhum reinício, travamento ou erro no journalctl
Reinício e queda de energia	Volta sozinho, com GPU, /srv e Docker funcionando

Se a temperatura ficar alta, aplicar o limite de potência da Etapa 5 e repetir as 2 horas de gpu-burn.

Entregável: relatório de testes com gpu.csv, saídas dos comandos e temperaturas máximas.

Etapa 8 – Monitoramento, contas da equipe de LLM e backup

Antes da entrega, a infraestrutura precisa ser avisada de qualquer problema no host, a equipe de LLM precisa conseguir entrar, e o host precisa ser reconstruível. Duração: um dia, em paralelo com as 24 horas da Etapa 7.

1. Exportadores de métricas

Rodam com network_mode: host, para que o ufw controle o acesso normalmente. Arquivo /srv/infra/monitoramento/docker-compose.yml:

```
services:
  node-exporter:
    image: prom/node-exporter:<versão>
    network_mode: host
    pid: host
    volumes:
      - /:/host:ro,rslave
    command: --path.rootfs=/host
    restart: unless-stopped
  dcgm-exporter:
    image: nvcr.io/nvidia/k8s/dcgm-exporter:<versão>
    network_mode: host
    cap_add: [SYS_ADMIN]
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    restart: unless-stopped

sudo ufw allow from <ip-do-prometheus> to any port 9100 proto tcp   # node-exporter
sudo ufw allow from <ip-do-prometheus> to any port 9400 proto tcp   # dcgm-exporter
```

No Prometheus (ou Zabbix) da empresa, cadastrar os dois alvos e os alertas:

Alerta	Condição sugerida
Host fora do ar	Sem resposta por 2 minutos
Temperatura da GPU	Acima de 85 °C por 5 minutos
Erro Xid da GPU	Qualquer ocorrência
Uso de disco em / ou /srv	Acima de 85%
SMART do NVMe	Alerta crítico ou desgaste acima de 80%
Nobreak	Operando na bateria

2. Nobreak

- Ligar o nobreak por USB e configurar o NUT (sudo apt install -y nut) para desligar o servidor com segurança quando a bateria estiver baixa.
- Testar tirando o nobreak da tomada por alguns minutos e conferindo o alerta.

3. Contas da equipe de LLM

```
sudo groupadd -f equipe-llm
```

```
# para cada integrante:
sudo adduser --disabled-password --gecos "" <usuario>
sudo usermod -aG equipe-llm,docker,ssh-users <usuario>
sudo install -d -m 700 -o <usuario> -g <usuario> /home/<usuario>/.ssh
echo "<chave-publica>" | sudo tee /home/<usuario>/.ssh/authorized_keys
sudo chown <usuario>: /home/<usuario>/.ssh/authorized_keys && sudo chmod 600
/home/<usuario>/.ssh/authorized_keys

# diretório de trabalho da equipe
sudo chown root:equipe-llm /srv/ia && sudo chmod 2775 /srv/ia
```

- O grupo docker equivale a acesso de administrador na máquina. Conceder só a quem vai operar containers, e registrar isso por escrito.
- Liberar no ufw as portas combinadas na Etapa 1, só para a rede interna.
- Manter uma lista de contas e o procedimento de revogação no desligamento de funcionários.

4. Backup do host

- O que entra: /etc, /srv/infra, a lista de pacotes e de pacotes fixados.
- O que não entra: pesos dos modelos e imagens Docker, que podem ser baixados de novo. O backup de /srv/ia é combinado com a equipe de LLM.

```
dpkg --get-selections > /srv/backup/pacotes.txt
apt-mark showhold > /srv/backup/pacotes-fixados.txt
```

- Ferramenta sugerida: restic enviando para o servidor de backup da empresa, agendado por systemd timer diariamente, com retenção de 7 diários e 4 semanais.
- Teste de restauração: reinstalar o host em uma VM seguindo este documento e o backup, medindo o tempo.

Entregável: métricas chegando ao Prometheus, alertas testados, nobreak integrado, contas criadas e testadas pelos próprios usuários, backup executado e restauração testada.

Etapa 9 – Entrega à equipe de LLM

A entrega é uma reunião de uma hora em que as duas equipes conferem o checklist juntas, na máquina. Depois dela, tudo que roda em containers é da equipe de LLM; o host continua com a infraestrutura.

1. Checklist de aceite (conferido pelas duas equipes)

- ☐ Um integrante da equipe de LLM entra por SSH com a própria chave
- ☐ O mesmo integrante roda `docker run --gpus all ... nvidia-smi` sem `sudo` e vê a GPU
- ☐ `/srv/ia` gravável pelo grupo `equipe-llm`, com espaço livre informado
- ☐ Versões de Ubuntu, kernel, driver, CUDA suportada, Docker e toolkit batem com o combinado
- ☐ Portas combinadas liberadas só para a rede interna
- ☐ Relatório da Etapa 7 aprovado e anexado
- ☐ Alertas de hardware chegando ao canal da infraestrutura
- ☐ Backup executado e restauração testada
- ☐ Reinício e queda de energia testados

2. Documentação entregue

- Este documento, atualizado com os valores reais (IPs, versões, nomes).
- Ficha técnica: componentes, versões de BIOS, kernel, driver, Docker e toolkit.
- Relatório de testes da Etapa 7.
- Runbooks curtos: reinstalar o host, trocar o driver, disco cheio, GPU travada ou com erro Xid, volta após queda de energia.
- Regras de convivência: portas publicadas só em `127.0.0.1` ou atrás de proxy; modelos em `/srv/ia`; nada instalado direto no host sem pedir à infraestrutura.

3. Divisão de responsabilidades

Assunto	Infraestrutura	Equipe de LLM
Sistema operacional e atualizações de segurança	Executa	É informada
Driver NVIDIA, Docker e toolkit (versões fixadas)	Executa em janela combinada	Solicita e valida
Firewall, portas e contas	Executa	Solicita
Monitoramento e alertas do host	Responde	É informada
Containers, modelos, vLLM, gateway	Não atua	Executa
Alertas dos serviços de IA	É informada	Responde
Reinícios da máquina	Agenda com a equipe de LLM	Confirma a janela

4. Suporte pós-entrega

- Duas semanas de acompanhamento próximo, com resposta no mesmo dia pelo canal da infraestrutura.

- Reunião curta ao fim desse período para fechar pendências.
- Depois disso, suporte normal, com janelas de manutenção agendadas.

Entregável: checklist assinado pelas duas equipes e documentação publicada.

Cronograma e riscos

Da preparação à entrega são oito dias úteis, seguidos de duas semanas de suporte. Os dois dias de testes de carga são quase só espera e correm junto com o monitoramento e as contas.

Etapa	Quando	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
1 – Preparação	dia 1	■																	
2 – Instalação do Ubuntu	dia 2		■																
3 – Configuração inicial	dia 3			■															
4 – Segurança e acesso	dia 4				■														
5 – Driver NVIDIA	dia 5					■													
6 – Docker e toolkit	dia 6						■												
7 – Testes de carga	dias 7 a 8							■	■										
8 – Monitoramento, contas, backup	dias 7 a 8							■	■										
9 – Entrega	**fim do dia 8**								◆										
Suporte pós-entrega	dias 9 a 18									■	■	■	■	■	■	■	■	■	■

Dias úteis. Barra azul = período da etapa; ◆ = entrega ao fim do dia 8. As Etapas 7 e 8 correm em paralelo.

Principais riscos

Risco	Impacto	Mitigação
Secure Boot impede o driver de carregar	GPU invisível após o reinício	Checagem da BIOS na Etapa 1; registro da chave MOK na Etapa 5
Atualização de kernel sem módulo NVIDIA compilado	GPU some após reinício	linux-headers instalados; conferir dkms status antes de qualquer reinício planejado
Atualização automática troca driver, Docker ou toolkit	Serviços de IA param	Pacotes fixados com apt-mark hold e excluídos do unattended-upgrades
Docker publica portas por cima do ufw	Serviço exposto na rede sem controle	"ip": "127.0.0.1" no daemon.json; exportadores em network_mode: host
Grupo docker dá acesso total à máquina	Alteração indevida no host	Conceder só a quem opera containers; registro por escrito; revisão periódica
Disco do sistema cheio por imagens e logs	Serviços param	Docker em /srv/docker; logs limitados; alerta em 85%
GPU superaquece sob carga contínua	Redução de desempenho ou desligamento	Teste de 24 h; limite de potência; alerta de temperatura
Queda de energia	Parada e risco de corromper dados	Nobreak com NUT; religamento automático testado
Conhecimento concentrado no executor	Suporte frágil	Revisor técnico em todas as etapas; runbooks publicados